



Escuela de Ingeniería en Computadores

CE3201 - Taller de Diseño Digital

Laboratorio #1 - Documento de Diseño

Estudiantes:

Mario Gudiño Rovira
Michael Valverde Navarro

Profesor:

Luis Alonso Barboza Artavia

Semestre:

II Semestre 2025

Índice

1. Repositorio en Github	2
2. Propuestas de Diseño	2
2.1. Problema 1	2
2.2. Problema 2	4
2.3. Problema 3	5

1. Repositorio en Github

Enlace al repositorio de Github:

https://github.com/hazielgr/m_gudino_mvalverde_digital_design_lab_2025

2. Propuestas de Diseño

En este documento se presentan las propuestas de diseño para cada problema correspondiente del laboratorio 1.

2.1. Problema 1

Para este problema se presenta la implementación de un conversor de un número binario de 4 bits en su equivalente en código Gray.

2.1.1. Método con compuerta XOR

El método utilizando XOR permite definir cada bit de salida, a partir del número binario de entrada, las ecuaciones booleanas corresponden a las siguientes:

$$G_3 = B_3, \quad (1)$$

$$G_2 = B_3 \oplus B_2, \quad (2)$$

$$G_1 = B_2 \oplus B_1, \quad (3)$$

$$G_0 = B_1 \oplus B_0. \quad (4)$$

A partir de las ecuaciones anteriores se definen las tablas de verdad para cada bit que componen a la salida.

Cuadro 1: $G_3 = B_3$

B_3	G_3
0	0
1	1

Cuadro 2: $G_2 = B_3 \oplus B_2$

B_3	B_2	G_2
0	0	0
0	1	1
1	0	1
1	1	0

Cuadro 3: $G_1 = B_2 \oplus B_1$

B_2	B_1	G_1
0	0	0
0	1	1
1	0	1
1	1	0

Cuadro 4: $G_0 = B_1 \oplus B_0$

B_1	B_0	G_0
0	0	0
0	1	1
1	0	1
1	1	0

2.1.2. Método con Sumas de Productos

Para este método cada compuerta XOR se intercambia por su equivalente en Suma de Productos, de manera que se utilizan solo compuertas más básicas para obtener el mismo resultado. Es decir:

$$A \oplus B = A\bar{B} + \bar{A}B$$

$$G_3 = B_3, \quad (5)$$

$$G_2 = B_3\bar{B}_2 + \bar{B}_3B_2, \quad (6)$$

$$G_1 = B_2\bar{B}_1 + \bar{B}_2B_1, \quad (7)$$

$$G_0 = B_1\bar{B}_0 + \bar{B}_1B_0. \quad (8)$$

Si se obtienen las tablas de verdad para estas ecuaciones booleanas se obtienen:

$$G_3 = B_3$$

B_3	G_3
0	0
1	1

$$G_2 = B_3\bar{B}_2 + \bar{B}_3B_2$$

$$G_1 = B_2\bar{B}_1 + \bar{B}_2B_1$$

$$G_0 = B_1\bar{B}_0 + \bar{B}_1B_0$$

B_3	B_2	$\overline{B_3}$	$\overline{B_2}$	$B_3\overline{B_2}$	$\overline{B_3}B_2$	G_2
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

B_2	B_1	$\overline{B_2}$	$\overline{B_1}$	$B_2\overline{B_1}$	$\overline{B_2}B_1$	G_1
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

2.1.3. Método seleccionado para el problema 1

Según las tablas de verdad se puede observar que el método más eficiente corresponde al método con compuertas XOR, ambos métodos son correctos y ambos métodos funcionan como se desean pero por simplicidad se decide desarrollar el problema 1 con el método de XOR. Se puede observar que hay muchos más pasos en el método de Sumas de Productos y es mejor compactar el trabajo para este problema.

2.2. Problema 2

En el desarrollo de este problema se presenta la implementación de un restador completo de 4 bits.

2.2.1. Método 1: “propagación anticipada de borrow” (Borrow Look-Ahead)

En lugar de propagar el borrow secuencialmente, se puede calcular el borrow de cada bit en paralelo usando lógica combinatorial derivada de los restadores de 1 bit. Esto sigue siendo un diseño estructural, porque cada bit sigue usando un módulo restador de 1 bit, pero las señales de borrow se calculan de manera anticipada.

Para cada bit i , el borrow $Bout_i$ depende de A_i , B_i y del borrow de entrada Bin_i . Calculamos una combinación lógica de estos para que el borrow del siguiente bit no tenga que esperar al anterior.

Cada módulo de 1 bit sigue siendo el mismo, pero conectamos la lógica de borrow anticipado para todas las etapas.

Se tiene el diagrama conceptual:

$A_0, B_0, Bin \rightarrow HalfSub \rightarrow D_0, Bout_0$

$A_1, B_1, Bout_0 \rightarrow FullSub \rightarrow D_1, Bout_1$

$A_2, B_2, Bout_1 \rightarrow FullSub \rightarrow D_2, Bout_2$

$A_3, B_3, Bout_2 \rightarrow FullSub \rightarrow D_3, Bout$

2.2.2. Método 2: Restadores de 1 bit conectados en cascada.)

Cada restador de 1 bit tiene entradas: A, B, Bin y salidas: $D, Bout$. La salida de borrow (Bout) de un bit se convierte en el borrow de entrada (Bin) del siguiente bit más significativo.

B_1	B_0	$\overline{B_1}$	$\overline{B_0}$	$B_1\overline{B_0}$	$\overline{B_1}B_0$	G_0
0	0	1	1	0	0	0
0	1	1	0	0	1	1
1	0	0	1	1	0	1
1	1	0	0	0	0	0

El resultado final es un vector de 4 bits D(3:0) y un Bout final. El Bout indica si en la resta hubo borrow es decir si el resultado es negativo.

Se tiene el diagrama conceptual:

A0,B0,Bin -> restador_1bit -> D0, b0

A1,B1,b0 -> restador_1bit -> D1, b1

A2,B2,b1 -> restador_1bit -> D2, b2

A3,B3,b2 -> restador_1bit -> D3, Bout

2.2.3. Tabla de verdad para restador de 1 bit

Para ambos diseños la tabla de verdad del restador de 1 bit es la misma:

A	B	Bin	D	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

2.2.4. Método seleccionado: Restadores de 1 bit conectados en cascada)

Se selecciona el método de restador en cascada de 4 bits para este proyecto porque es la primera vez que se trabaja con la herramienta Quartus y con los lenguajes de descripción de hardware, y este enfoque resulta más educativo y fácil de comprender a nivel conceptual. Al construir el restador a partir de módulos de 1 bit conectados secuencialmente, se puede visualizar claramente cómo se propaga el borrow de un bit al siguiente, lo cual facilita el entendimiento de la operación de resta binaria. Además, este método es ampliamente documentado y existen numerosos ejemplos y recursos en internet, lo que permite acceder fácilmente a información adicional y comparaciones durante el desarrollo. Además realizando la lectura del capítulo 4 de libro Harris&Harris se obtiene un ejemplo de un sumador completo de 1 bit, se toma este de ejemplo para realizar este desarrollo.

2.3. Problema 3

Para este problema se presenta la implementación de un contador parametrizable con N bits, con funciones de carga y reset.

2.3.1. Contador binario parametrizable (Opción A)

Un contador binario de N bits es un circuito secuencial síncrono que incrementa su valor en una unidad cada ciclo de reloj. El rango de conteo es de 0 a $2^N - 1$, realizando un reset automáticamente cuando se alcanza el valor máximo. Este contador incluye señales adicionales que permiten controlar su comportamiento:

- **Reset asíncrono** ($arst_n$): fuerza el valor del contador a cero de manera inmediata, sin necesidad de esperar un flanco de reloj.
- **Load**: permite cargar en el contador un valor específico L proveniente de los *switches* de la FPGA.
- **Enable**: controla si el contador avanza ($enable = 1$) o se mantiene en su valor actual ($enable = 0$).

La ecuación de próximo estado se expresa como:

$$q^+ = \begin{cases} 0 & \text{si } \neg arst_n \\ L & \text{si } arst_n \wedge load \\ (q + 1) \text{ mód } 2^N & \text{si } arst_n \wedge \neg load \wedge enable \\ q & \text{en otro caso} \end{cases}$$

donde:

- q es el valor actual del contador.
- q^+ es el próximo estado.
- L es el valor cargado desde los *switches*.
- N es el número de bits del contador.

la implementación puede detallarse bit a bit mediante una suma por *ripple-carry*. Se definen las señales de acarreo como:

$$c_0 = enable, \quad c_{i+1} = q_i \cdot c_i$$

y la suma parcial para cada bit:

$$s_i = q_i \oplus c_i$$

donde:

- q_i es el bit actual del contador.
- s_i es el resultado de la operación de suma con acarreo.

Este diseño es escalable: para $N = 2$ el rango de conteo es 0 a 3, para $N = 4$ el rango es 0 a 15 y para $N = 6$ el rango es 0 a 63.

2.3.2. Contador binario como máquina de estados (Opción B)

Otra forma de implementar un contador binario de N bits es mediante una máquina de estados finitos. Cada valor posible del contador se modela como un estado de la máquina, y las transiciones entre estados representan la operación de conteo.

El conjunto de estados está dado por:

$$S = \{0, 1, 2, \dots, 2^N - 1\}$$

La máquina tiene dos entradas principales:

- **Enable:** permite avanzar al siguiente estado cuando está en 1.
- **Load:** carga un valor específico L en el estado actual.

Además, cuenta con un reset asíncrono ($arst_n$), que fuerza a la FSM al estado inicial 0 sin necesidad de reloj.

El comportamiento de la máquina puede expresarse como:

$$q^+ = \begin{cases} 0 & \text{si } \neg arst_n \\ L & \text{si } arst_n \wedge load \\ \delta(q) & \text{si } arst_n \wedge \neg load \wedge enable \\ q & \text{en otro caso} \end{cases}$$

donde la función de transición de estados δ está definida como:

$$\delta(q) = \begin{cases} q + 1 & \text{si } q < 2^N - 1 \\ 0 & \text{si } q = 2^N - 1 \end{cases}$$

Es decir, la máquina avanza secuencialmente por los estados de 0 a $2^N - 1$ y, al alcanzar el máximo, vuelve al estado 0 de forma automática

2.3.3. Método seleccionado para el problema 3

Se escoge la opción A, pues basta con usar registros (flip-flops) y un sumador que incremente en uno. La opción A permite cambiar el ancho del contador modificando solo un parámetro (N) por lo que la opción A resulta ser más clara y concisa. En la opción B se tendrían que definir todos los estados y las transiciones cada vez que cambie N lo cual generaría problemas de escalamiento